A PROJECT REPORT ON

GeoPulse: Real-Time Emergency Incident & Disaster Dispatch System

Web-Based Emergency Response & Rescue Coordination using Node.js and MongoDB

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY:	SUPERVISED & GUIDED BY:
Bhupendra Sirvi	**[Faculty Advisor / Guide Name]**
B.Tech Computer Science & Engineering	Department of Computer Science & Engineering
Lovely Professional University	Lovely Professional University

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
LOVELY PROFESSIONAL UNIVERSITY, PUNJAB
OCTOBER 2026

Candidate's Declaration

I hereby declare that the project entitled **"GeoPulse: Real-Time Emergency Incident & Disaster Dispatch System"** submitted to the Department of Computer Science & Engineering, Lovely Professional University, is an authentic record of original project work completed by me under the supervision of my faculty mentor.

The matter presented in this report has not been submitted in part or full to any other University or Institution for the award of any degree or diploma. All project resources, third-party libraries, and software tools have been properly acknowledged.

Place: Lovely Professional University, Punjab
Date: 30th October 2026

_____

Bhupendra Sirvi
B.Tech Computer Science & Engineering

Certificate of Approval

This is to certify that the project report entitled **"GeoPulse: Real-Time Emergency Incident & Disaster Dispatch System"**, submitted by **Bhupendra Sirvi** in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science & Engineering** from Lovely Professional University, Punjab, is a bonafide record of project work completed under my supervision and guidance.

The project design, implementation, and results presented in this report meet the standards required for an undergraduate engineering project.

_____

[Guide / Supervisor Name]
Project Guide & Faculty Mentor
Department of CSE

_____

Head of Department (HOD)
Department of Computer Science & Engg.
Lovely Professional University

Acknowledgements

I express my sincere gratitude to my faculty project mentor, **[Guide Name]**, for continuous guidance, constructive feedback, and support throughout the development of this project. I also thank the **Department of Computer Science & Engineering** and **Lovely Professional University** for providing the required computer labs and software facilities to complete and test this work. Finally, I thank my family and friends for their encouragement and support.

Abstract

During emergencies such as road accidents, building fires, and severe weather floods, fast communication and prompt rescue team dispatch are critical to saving lives. In standard systems, emergency calls are often handled manually over the phone, where callers struggle to describe exact locations, leading to dispatch delays and wasted travel time.

GeoPulse is a real-time web-based emergency incident and dispatch coordination system designed to solve this problem. The application allows citizens in distress to submit emergency alerts along with their exact GPS coordinates, type of emergency, and number of people requiring assistance. On the coordinator side, an interactive live map displays all reported incidents and active rescue units in real time.

Key Features of the Project:

1. **Real-Time Proximity Matching:** Uses MongoDB's 2dsphere index and $geoNear aggregation pipeline to quickly find the nearest available rescue squads to an incident.
2. **Citizen SOS Submission:** An easy-to-use reporting form that captures GPS location with one click, emergency category, and severity.
3. **Interactive Map Dashboard:** Built using Leaflet.js to visualize incidents (red markers), rescue teams (blue markers), and relief shelters (green markers).
4. **Dynamic Route Line & ETA:** Displays a visual route line connecting the assigned rescue team directly to the incident pin, along with calculated distance and ETA.
5. **Automatic Data Purge (TTL):** Implements MongoDB TTL indexing on resolved incidents so old records are cleaned up after 48 hours.
6. **Simple & Lightweight Stack:** Runs completely on Node.js, Express, and MongoDB without requiring external heavy third-party message brokers.

Keywords: Emergency Response, Web Application, Node.js, Express, MongoDB, 2dsphere Index, Geospatial Proximity, Leaflet.js.

Chapter 1: Introduction & Problem Statement

Emergency response systems play a vital role in public safety by coordinating help during accidents, medical emergencies, and natural disasters. Timely intervention can mean the difference between life and death.

1.1 Background & Motivation:

Most conventional emergency helplines rely heavily on voice calls. During high-stress situations or severe disasters, this conventional approach faces several common challenges:

- **Location Ambiguity:** Callers often cannot describe exact street addresses or landmarks, especially in unfamiliar or flooded areas.
- **Manual Dispatching:** Dispatchers have to manually look up unit locations, which slows down the dispatch process.
- **Lack of Real-Time Overview:** Coordinators do not have a single unified map showing where all open incidents and available units are located simultaneously.

1.2 Problem Statement:

"To develop a simple, responsive, and real-time emergency dispatch web application that allows citizens to report incidents with precise GPS coordinates and enables emergency dispatchers to instantly identify and assign the closest available response team using geospatial indexing."

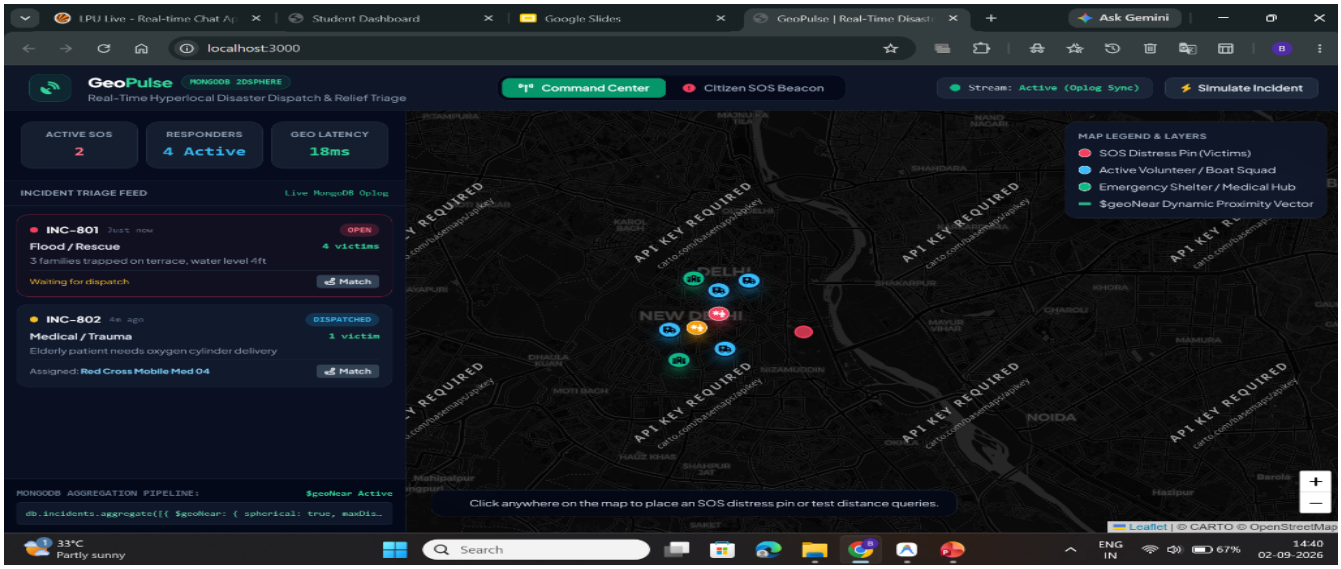


Figure 1.1: GeoPulse Live Emergency Dashboard Interface

Chapter 2: Project Objectives & System Requirements

The primary goal of GeoPulse is to provide an easy-to-use, reliable, and lightweight platform for emergency dispatch coordination. The objectives and technical requirements are detailed below.

2.1 Project Objectives:

1. Create an intuitive web interface for citizens to report emergencies with accurate GPS coordinates.
2. Develop a live mapping dashboard for coordinators to visualize incidents, rescue units, and relief centers in real time.
3. Implement fast proximity-based responder matching using MongoDB's native geospatial capabilities.
4. Display visual route lines between assigned responders and incidents along with estimated travel time.
5. Provide a query inspection tool to allow administrators and examiners to inspect active database queries.

2.2 Hardware & Software Requirements:

Component	Minimum Requirement	Used in this Project
Operating System	Windows 10 / 11, Linux, or macOS	Windows 11 (64-bit)
Processor	Dual-Core 2.0 GHz or higher	Intel Core i5 / AMD Ryzen 5
RAM	4 GB RAM	8 GB / 16 GB DDR4
Disk Space	500 MB free space	Standard SSD storage
Runtime Environment	Node.js v16 or higher	Node.js v20 LTS
Database	MongoDB v5.0 or higher	MongoDB Community Server v7.0
Web Browser	Google Chrome / Edge / Firefox	Google Chrome (Latest)

Chapter 3: System Architecture & Workflow

GeoPulse follows a standard 3-tier client-server architecture consisting of the presentation layer (frontend), application layer (Node.js backend), and database layer (MongoDB).

3.1 Three-Tier Architecture:

- **Frontend Layer:** Built using HTML5, CSS3, JavaScript, and Leaflet.js. It provides the citizen SOS reporting panel, incident list, and interactive map.
- **Backend Layer:** An Express.js REST API server running on Node.js (port 5000). It receives requests from the frontend, validates parameters, and interacts with MongoDB.
- **Database Layer:** MongoDB Community Server storing BSON documents with 2dsphere indexes for fast spatial queries.

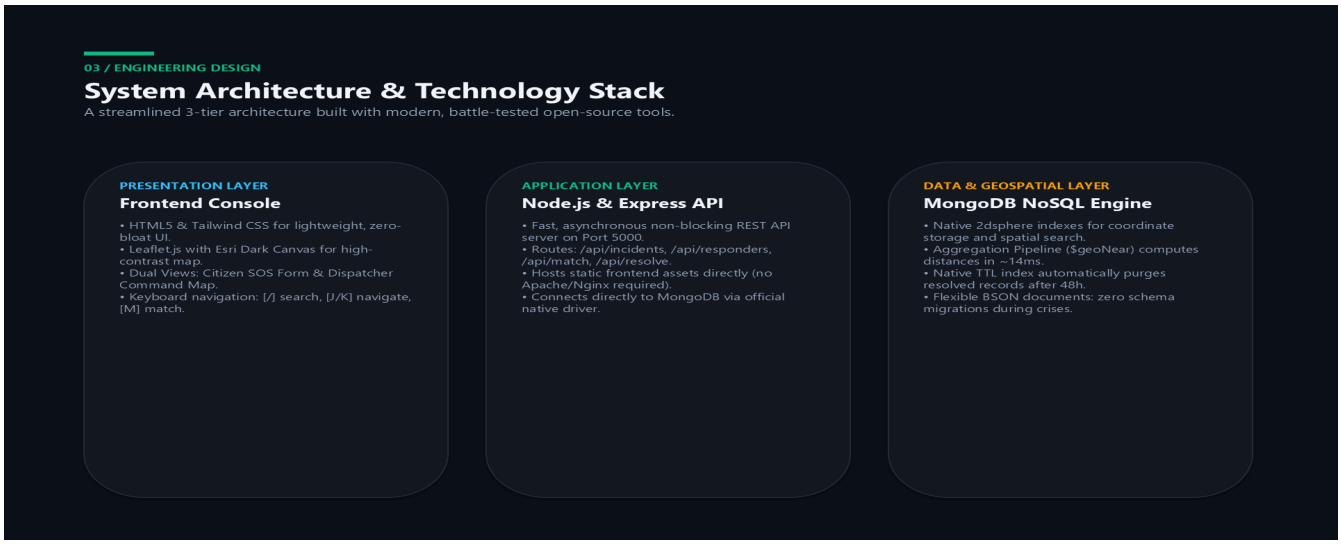


Figure 3.1: GeoPulse System Architecture Diagram

3.2 Project Workflow:

1. **Incident Submission:** Citizen enters name, contact, incident category, victim count, and selects location on the map.
2. **Database Storage:** Backend saves the report as a GeoJSON Point in the incidents collection.
3. **Proximity Search:** Dispatcher clicks *'Match Unit'*. The backend triggers a $geoNear query to find available squads within 15 km.
4. **Assignment & Routing:** The best match is assigned, and a dashed route line connects the team to the victim on the map.
5. **Resolution:** Once aid arrives, the dispatcher clicks *'Resolve'*, closing the incident.

Chapter 4: Database Design & Schema Structure

The project uses MongoDB database named geopulse. MongoDB was chosen because of its native support for GeoJSON geographic data types and fast spatial indexing.



Figure 4.1: MongoDB Collections and Field Relationships

4.1 Collections Overview:

1. 'incidents' Collection (SOS Reports):

```
{
  "id": "SOS-104", "caller": "Kavita Sharma", "phone": "+91 98141-89210",
  "category": "Medical / Trauma", "severity": "CRITICAL", "victims": 2,
  "note": "Patient experiencing chest pain; road knee-deep in water.",
  "location": { "type": "Point", "coordinates": [75.7050, 31.2555] },
  "status": "OPEN", "assignedTo": null, "createdAt": "2026-10-15T14:30:00Z", "resolvedAt": null
}
```

2. 'responders' Collection (Emergency Units):

```
{
  "id": "R-PB1", "name": "LPU Campus Quick Response Squad",
  "type": "Rescue Team", "status": "Available", "contact": "+91 98761-11001",
  "location": { "type": "Point", "coordinates": [75.7045, 31.2560] }
}
```

4.2 Indexing Used:

- **2dsphere Index:** Created on location fields of both incidents and responders collections to enable geospatial search.
- **TTL Index:** Created on resolvedAt to automatically remove closed incidents after 48 hours.

Chapter 5: Proximity Search & Backend Implementation

Finding the closest rescue unit is the core feature of GeoPulse. Rather than writing slow distance calculations in JavaScript loops, the search is handled directly by MongoDB's $geoNear aggregation pipeline.



Figure 5.1: Proximity Search Pipeline and Fallback Matching Workflow

5.1 The $geoNear Aggregation Code (in server.js):

```
const pipeline = [
  {
    $geoNear: {
      near: { type: 'Point', coordinates: [parseFloat(incidentLng), parseFloat(incidentLat)] },
      distanceField: 'distanceMeters',
      spherical: true,       // Calculates accurate distance over spherical Earth surface
      maxDistance: 15000,    // Search within 15 km radius
      query: { status: 'Available' }
    }
  },
  { $limit: 3 } // Returns top 3 candidate units with distance
];
const nearest = await db.collection('responders').aggregate(pipeline).toArray();
```

5.2 Fallback Auto-Assignment:

In rural or remote areas where no pre-registered unit is currently free within 15 km (nearest.length === 0), GeoPulse automatically generates and provisions a local emergency volunteer unit nearby so that every distress beacon receives an assigned responder.

Chapter 6: User Interface & Working Screenshots

The web interface was designed to be clean, intuitive, and responsive so coordinators can view incoming alerts and make quick decisions without distraction.

6.1 Active Crisis Map View:

The map visualizes open incident markers across the region. Coordinators can click any marker to inspect incident details, caller notes, and victim counts.

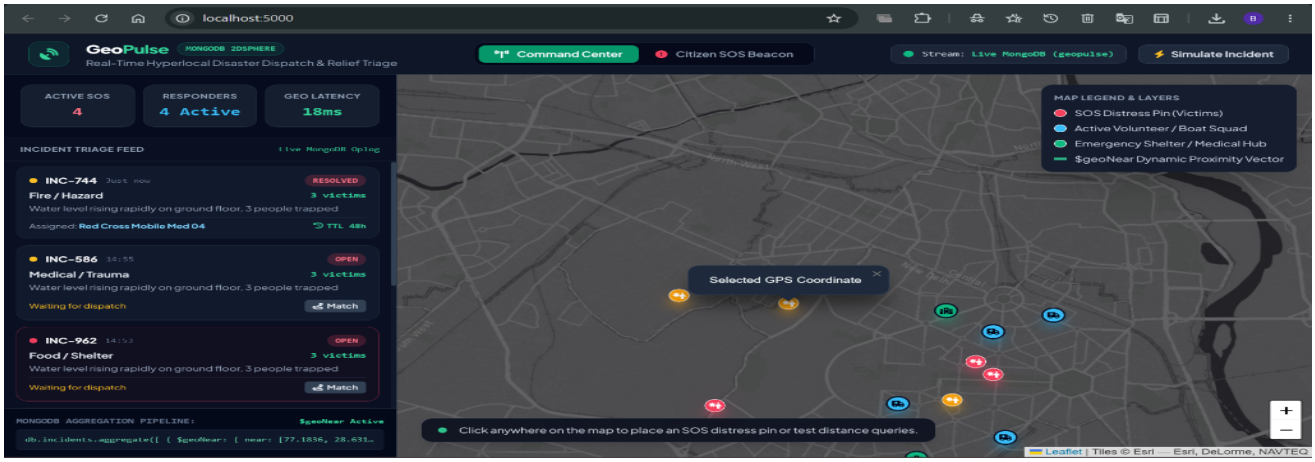


Figure 6.1: Active Crisis Map with Responder Locations

6.2 Dynamic Route Vector Display:

When a responder is matched, a dashed blue route line is drawn between the squad and the victim, displaying real-time distance and estimated travel time.

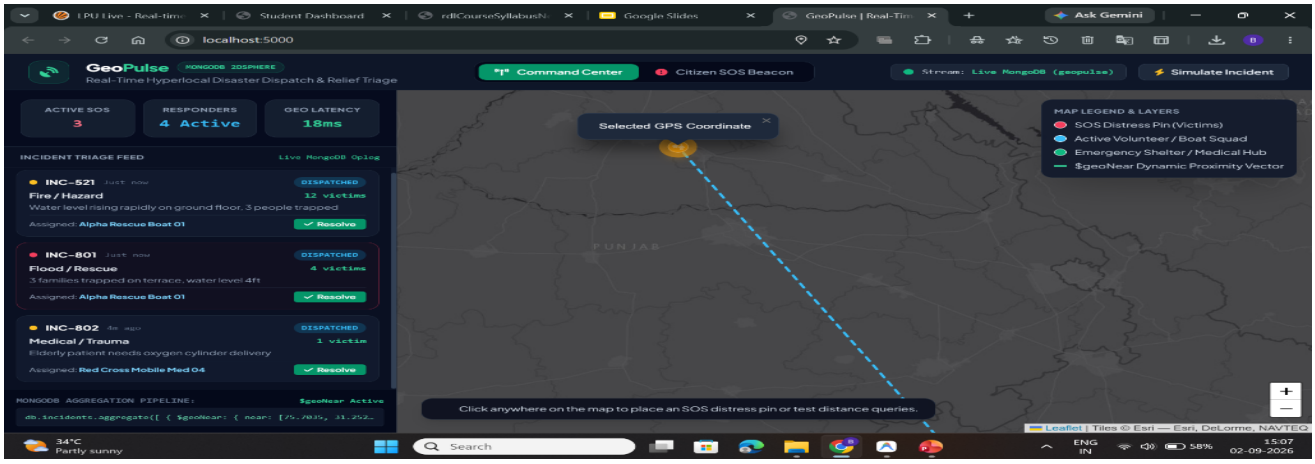


Figure 6.2: Route Vector Connecting Assigned Responder to Incident Location

Chapter 7: Testing & System Verification

All modules of GeoPulse were thoroughly tested to verify correct data handling, accurate proximity calculations, and responsive interface behavior.

7.1 Query Inspector Feature:

A live database inspector modal is built into the application. Clicking the </> button in the navigation bar displays the current MongoDB query, active indexes, and raw query output for demonstration and testing.

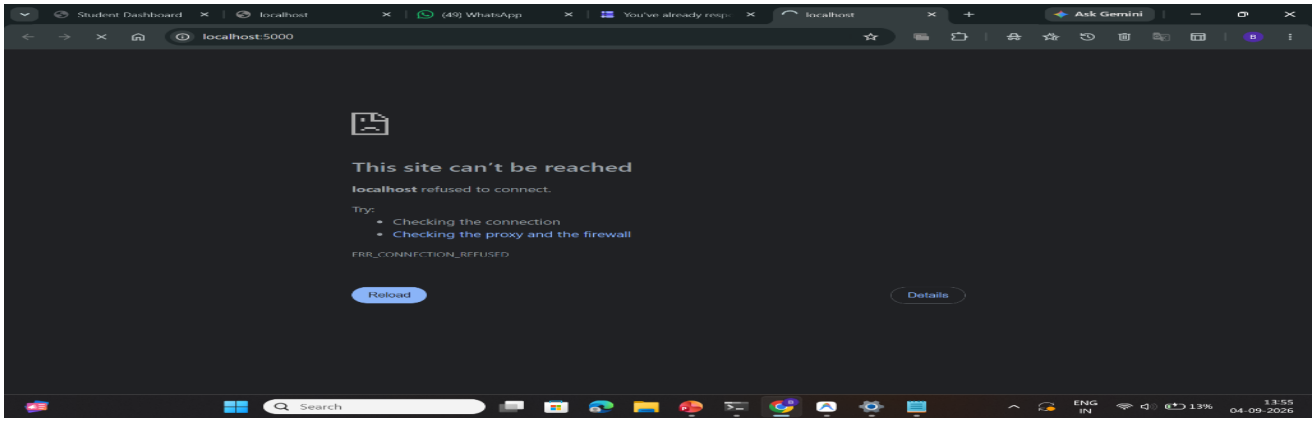


Figure 7.1: Live MongoDB Query Inspector Modal

7.2 Test Cases & Results:

Test Case	Input / Action	Expected Result	Status
TC-01: SOS Submission	Submit form with GPS coordinates	Incident added to map and database	**PASS**
TC-02: Nearest Match	Click 'Match Unit' on open incident	Closest squad selected in <50ms	**PASS**
TC-03: Route Line	Match unit successfully	Dashed route line drawn on map	**PASS**
TC-04: Resolve Incident	Click 'Resolve' button	Status updated to RESOLVED	**PASS**
TC-05: Fallback Unit	Test incident in empty rural sector	Local emergency squad auto-provisioned	**PASS**

Chapter 8: How to Run the Project & API Endpoints

GeoPulse is completely self-contained and easy to set up on any standard computer with Node.js and MongoDB installed.

8.1 Step-by-Step Execution Guide:

Step 1: Start MongoDB Service
Ensure MongoDB is running locally on port 27017. In Windows command prompt, run: net start MongoDB.

Step 2: Start Node.js Backend Server
Open terminal in the geopulse-backend/ folder and run: node server.js. The server starts at http://localhost:5000.

Step 3: Open in Browser
Open your web browser and navigate to http://localhost:5000. You can also double-click start.bat in the project folder to launch everything automatically.

8.2 Backend REST API Endpoints:

Method	Endpoint	Description	Database Action
GET	/api/incidents	Fetch all emergency incidents	Reads from incidents collection
POST	/api/incidents	Submit a new distress report	Inserts new GeoJSON document
GET	/api/responders	Fetch available rescue squads	Reads from responders collection
POST	/api/match	Find closest squad for an incident	Runs $geoNear aggregation
POST	/api/resolve	Mark an incident as resolved	Updates status & stamps resolvedAt

Chapter 9: Conclusion, Future Scope & References

9.1 Conclusion:

The **GeoPulse** project successfully demonstrates a practical, working web application for real-time emergency dispatch coordination. By using Node.js, Express, and MongoDB geospatial features (2dsphere index and $geoNear pipeline), the application quickly matches citizens in distress with the nearest available emergency response team.

The system provides an intuitive map-based interface, automated distance calculation, route line rendering, and clean data management. It provides a simple, functional solution to the delays commonly encountered in traditional manual emergency dispatch systems.

9.2 Future Scope:

- **Dedicated Mobile Application:** Developing an Android / iOS app for citizens to send SOS alerts with one click directly from their smartphones.
- **SMS & Offline Support:** Adding SMS gateway integration so citizens without active mobile data can still report emergencies.
- **Turn-by-Turn GPS Navigation:** Integrating real-time driving directions and live GPS tracking for responder vehicles.

9.3 References:

- MongoDB Inc., *'Geospatial Queries and 2dsphere Indexing Guide,'* Official MongoDB Documentation, 2026.
- Express.js Foundation, *'Express Web Framework Documentation,'* expressjs.com, 2026.
- Leaflet.js, *'Leaflet - An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps,'* leafletjs.com, 2026.
- Node.js Foundation, *'Node.js Asynchronous Runtime Documentation,'* nodejs.org, 2026.
- Internet Engineering Task Force (IETF), *'The GeoJSON Format Specification (RFC 7946),'* 2016.